

COMP 577: HOMEWORK 1

PANORAMA STITCHING

September 2, 2026

Homework 1 is graded out of 60 points and contributes 10% towards your final score.

In this homework project, you will build a pipeline to stitch several images into one panorama. You will also capture a set of your own images to report the final results. This project focuses on building a Computer Vision pipeline from existing pieces. Don't implement low-level functionality unless you really want to! It's best to use OpenCV's Python package **cv2**.

You are encouraged to review Section 8 of the book [1].

Submission Instructions

- Upload a .pdf report for the project. This report should explain all 5 steps of panorama building. You NEED to show input and output for every step.
- Upload .py files for every piece of code you wrote for this assignment. Comment the code reasonably well. Don't upload a .zip file with all codes; just upload a single .py or .pynb file.

Step 1. Feature detection and description (5 points). The first step in this project is to perform feature detection individually on each image in the sequence. Recall some feature detectors that we covered in the class: Harris Corner, Blob Detector, SIFT Detector, etc. Coupled with the detection step goes feature description. Your features have to be invariant to some transformations. Think about the way you'll collect the dataset. What kind of invariance do you need? Explore the feature descriptors implementation available in OpenCV: SIFT, SURF, ORB, etc.

1. Read the input images using **cv2.imread**.
2. Convert your images to grayscale (e.g. use **cv2.cvtColor**) for feature extraction. You may also resize your images (e.g. to a height of 480 pixels as in the reference *yosemite* sequence) via **cv2.resize** to make the following steps run faster.
3. Pick your feature **detector**. A good place to start is **cv2.SIFT_create** (or **cv2.xfeatures2d.SIFT_create** in older OpenCV versions).
4. Use **detectAndCompute** method to extract the keypoints and keypoint features.

Figure 1 visualizes the detected keypoints using **cv2.drawKeypoints**.

Step 2. Feature matching and image alignment (10 points). Once the keypoints and feature are computed, your code needs to establish good matches between consecutive pairs of images. For that, you need to pick a distance function and a matching algorithm.

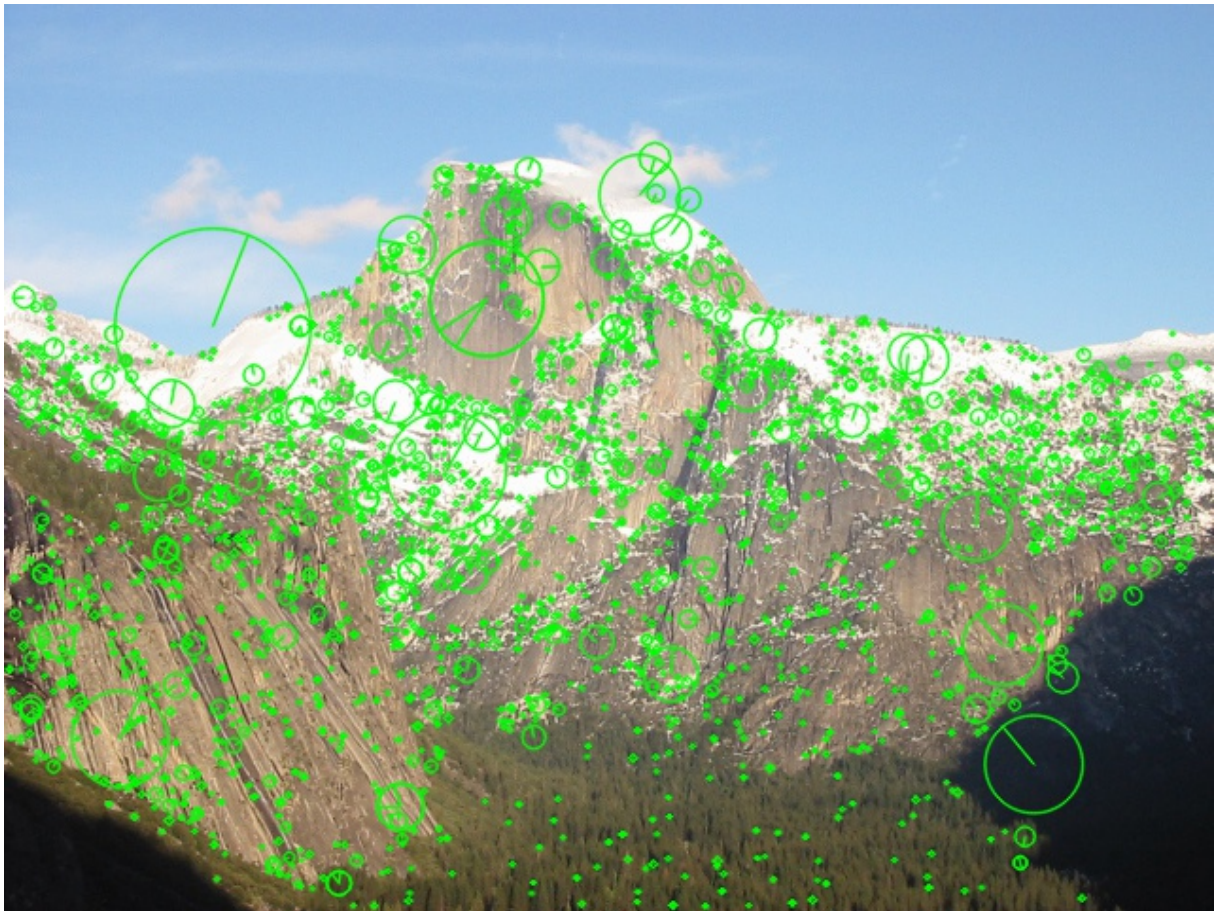


Figure 1: SIFT keypoints

1. Pick a matching algorithm and a distance function. Brute force matching (**cv2.BFMatcher**) can do the job, as you will later get rid of the bad matches! If you used binary features, such as BRIEF or ORB, look at the hamming distance function (**cv2.NORM_HAMMING**), otherwise it suffices to just use the Euclidean distance (**cv2.NORM_L2**).
2. Use the **match()** method to compute matches from a pair of images, providing their features as an input. Pick a pair of images that has a good overlap. In fact, it is convenient to capture the images for panorama stitching in some pre-defined order (e.g. from left to right) so that adjacent images always have good overlap.
3. Using original keypoint positions, and the matches, construct the correspondences in a form of two $m \times 2$ arrays **pts1** and **pts2**, such that **pts1[i]** and **pts2[i]** store the positions of the match in the first and second image, respectively. If you're using the suggested OpenCV functionality, look at the interfaces for the KeyPoint and DMatch classes to understand how to manipulate the return objects.
4. Now, estimate the homography transformation between the images from the correspondences. The matches will have a lot of outliers, so you need to pick an estimator that is robust to outliers, such as RANSAC. **cv2.findHomography** is useful for this step!
5. Finally, warp one of the images with the obtained homography: **cv2.warpPerspective**.

Figure 2 shows top-10 matches (based on the response score) of the 1249 matches detected between the chosen pair of images. The figure was created using `cv2.drawMatches`. Take a look at Figure 3 and Figure 4. Why is alignment in the former so successful? Why are there issues in the latter figure?

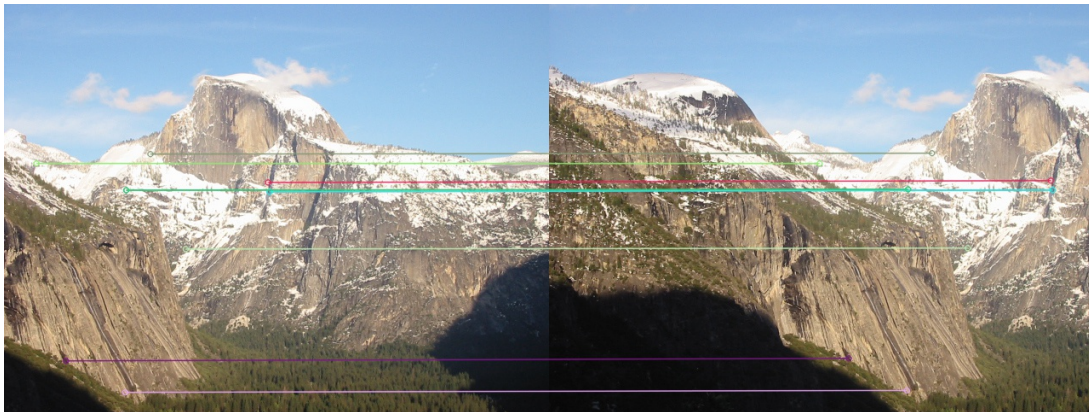


Figure 2: Top 10 of 1249 matches between a pair of images

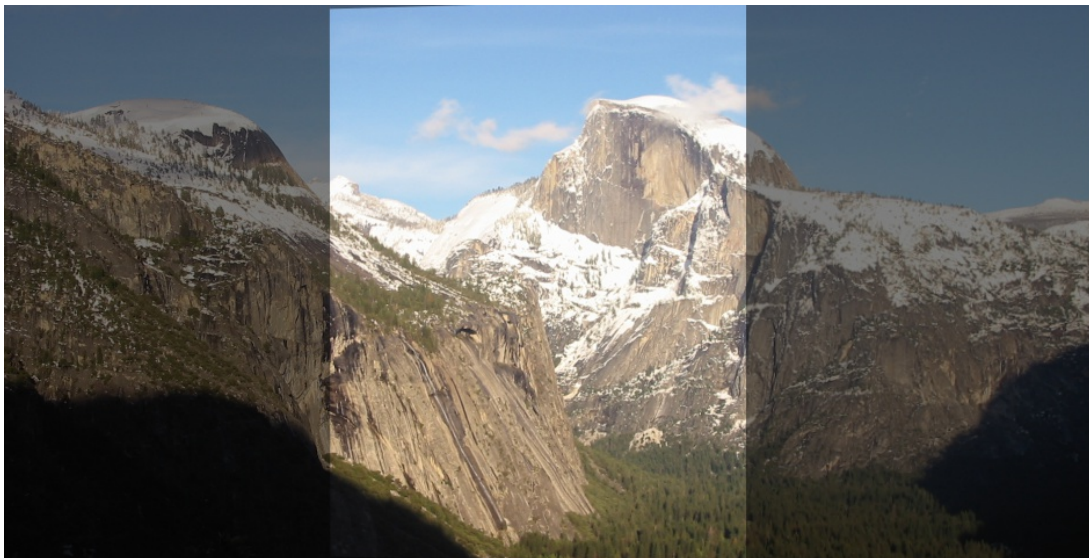


Figure 3: Adding intensities of two images: the original left image, and the right image warped onto the plane of the left image.

Step 3. Image blending (5 points). Now that you have the original image1, and image2 warped onto the plane of image1, you need to combine them to produce a seamless result. If you simply add the image intensities, the overlapping region would look good, but other regions would seem dark, as Figure 3 shows. Another straightforward solution is copy-pasting one image on top of the other. This would also create artifacts on the boundaries, as in Figure 5.

One solution to reduce the intensity of the artifacts is to blend the images using adaptive weights. The pixels close to the boundary of each image are less reliable than the ones in the center. How about blending the pixels with the weights that are proportional to the distance of those pixels to the corresponding image boundary? We provide the `get_distance_transform` function that takes a warped RGB image $H \times W \times 3$



Figure 4: A challenging case. Notice the artifacts on the chimneys, even though the facade is well-aligned. The camera is relatively close to the building, and has a wide baseline and a rotation between the images. The proper warping is more complex than a simple homography for these images.

and produces a single-channel distance function $H \times W \times 1$. You can use this function to compute the result as `img1 * weight1 + img2 * weight2`. Don't forget to normalize the resulting image to the $0 \dots 255$ range. Try using `np.maximum(denom, 1.0)` to avoid division by zero. Convert your image back to unsigned 8 bit integer with `img.astype("uint8")`.

```

1
2 def get_distance_transform(img_rgb):
3     """
4     Get distance to the closest background pixel for an RGB image
5     Input:
6         img_rgb: np.array, HxWx3 RGB image
7     Output:
8         dist: np.array, HxWx1 distance image
9             each pixel's intensity is proportional to
10            its distance to the closest background pixel
11            scaled to [0..255] for plotting
12     """
13     # Threshold the image: any value above 0 maps into 255
14     thresh = cv2.threshold(img_rgb, 0, 255, cv2.THRESH_BINARY)[1]
15     # Collapse the color dimension
16     thresh = thresh.any(axis=2)
17     # Pad to make sure the border is treated as background
18     thresh = np.pad(thresh, 1)
19     # Get distance transform
20     dist = distance_transform_edt(thresh)[1:-1, 1:-1]
21     # HxW -> HxWx1
22     dist = dist[:, :, None]
23     return dist / dist.max() * 255.0

```

Step 4. Stitching multiple images (10 points). One simple approach to stitching n images is to sort your sequence from left to right, and progressively apply pairwise stitching: if you stitch each pair of adjacent images, you get $n - 1$ resulting panoramas. You can apply the same stitching process to those images again and again, until you are left with the single wide panorama. Figure 7 shows the result of

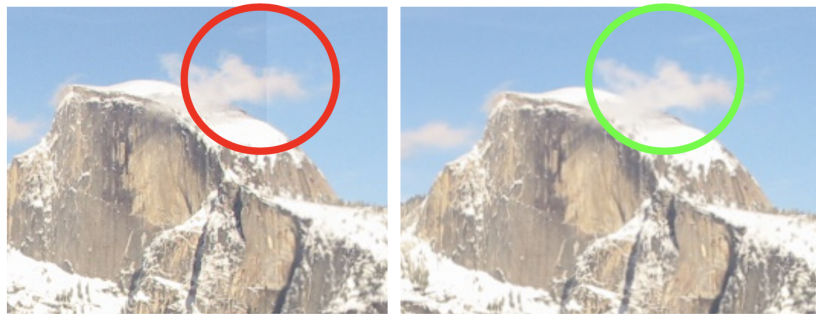


Figure 5: (Left): Artifacts from simple copy-pasting the left image on top of the warped right image. (Right): linear blending (see Figure 6).



Figure 6: Blending: every pixel from the corresponding image contributes with a weight proportional to its distance to the boundary of that image.

such progressive stitching.

This is a very inefficient approach, as it needs $O(n^2)$ runs of the pairwise functions. However, it can help you organize your code from Steps 1-3 into functions, after which you simply need to add one **for** loop on top of it.

You are welcome to try a more efficient stitching, such as estimating pairwise homographies $H_{1 \rightarrow 2}$ and $H_{2 \rightarrow 3}$ and warping image 1 onto the plane of image 3 using superposition: $H_{1 \rightarrow 3} = H_{2 \rightarrow 3} H_{1 \rightarrow 2}$. For example, you can choose the middle image as a common anchor to align all other images to. In this case, discuss how the error (drift) accumulates through the superposition.

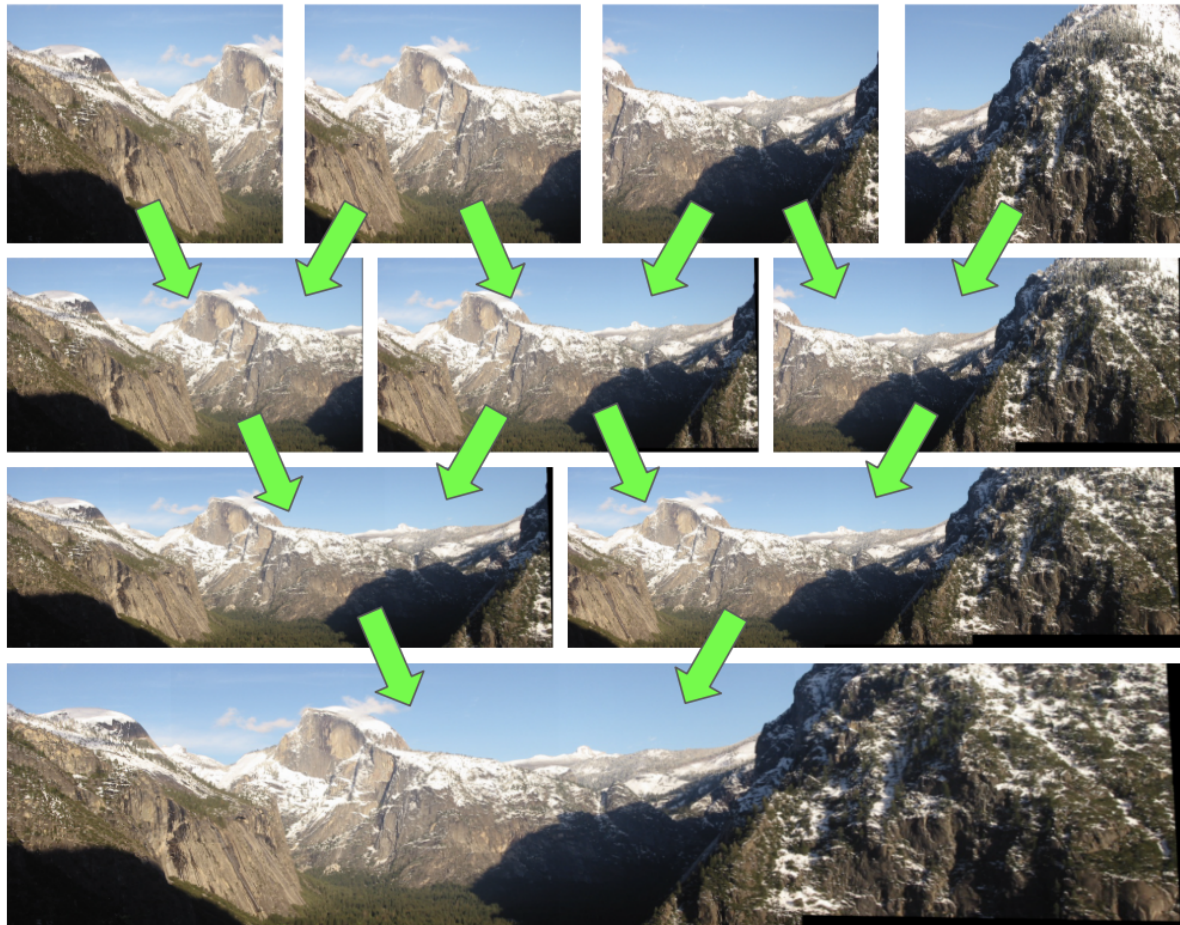


Figure 7: Progressive pairwise blending. Images are sorted from left to right, and stitched in pairs. The resulting stitched images undergo the same process til getting the resulting panorama.

Step 5. Create your own panorama and discuss the setup and the results (30 points). For this part, go shoot a short photo sequence, run your stitching pipeline, and explain what happened. There are a few things to consider while capturing your own set of images in order to get a good panorama.

Some Suggestions: Capture several images (at least 4-5). If you work with just 2-3 images, or images with very high overlap, you won't be able to gain good insights into the sources of errors that panoramas experience.

It's desirable to expose the errors of your pipeline for the sake of this homework. That makes your understanding deeper and the discussion you provide more valuable. The neighboring images in your sequence must have good overlap in order to produce robust alignment.

Your panorama stitching task still has to be somewhat challenging. Don't capture two images of a flat surface shifting your camera in a parallel plane by one inch! Try some nontrivial camera baselines and rotations, but mind your answer to the previous bullet.

Complete the following steps:

- (5 points) The input images and the final panorama with your original pipeline.
- (5 points) A separate copy of the panorama with all errors clearly marked. Clearly mark the errors with red arrows or boxes.

- (10 points) For each error you point out, describe in a few words what part of the pipeline likely causes this issue and why. For example, if you have rotated the camera a lot, and two images have a small overlap, you will likely have few good matches and hence have large errors in homography estimation, resulting in visible misalignment between the pictures. Just providing an explanation is not enough; you want to provide concrete evidence. For example, show how the number of good matches decreases in your panorama vs the Yosemite example.
- (10 points) For each error you pointed out, how would you resolve this issue? You **MUST** modify the pipeline (e.g., changing parameters) or change the capture setup. Please include what you changed and show the new panorama. If you re-capture the same scene, please discuss how the setup is different. If you can not fix the problem, discuss why a simple modification of the pipeline or capture setup is not enough to improve the panorama generation.

You will need to point out at least 1 major error in your panorama. If you get a perfect panorama, get adventurous and capture harder scenes with challenging camera motion. If you are unsure, please ask the TA.

References

[1] Richard Szeliski. *Computer vision: algorithms and applications*. Springer Nature, 2022.